

PRÁTICA 4 – Conversor analógico digital

1. Introdução

O método de aproximação sucessiva é uma técnica comumente utilizada em conversores analógico-digitais (ADCs) para converter um sinal analógico em um valor digital correspondente. O processo é feito em etapas sucessivas, onde a precisão da conversão é aprimorada a cada passo.

O ADC começa por comparar a tensão do sinal analógico de entrada com uma tensão de referência. Com base nesta comparação, o ADC determina se o sinal de entrada é maior ou menor do que a tensão de referência e armazena este resultado em um bit. Em seguida, o ADC atualiza a tensão de referência, tornando-a mais próxima do valor do sinal analógico de entrada. Este processo é repetido várias vezes com cada vez mais bits sendo adicionados, até que se obtenha a precisão desejada.

O processo de aproximação sucessiva é bastante eficiente, pois a cada passo a precisão é aumentada, evitando erros de arredondamento. No entanto, o tempo de conversão é diretamente proporcional ao número de bits utilizados, o que pode ser um problema em sistemas que precisam de altas taxas de amostragem. Para resolver este problema, muitos ADCs utilizam técnicas como a subamostragem para aumentar a velocidade de conversão sem comprometer a precisão. A Figura 1 ilustra o diagrama de blocos de um circuito conversor AD usando aproximação sucessivas.

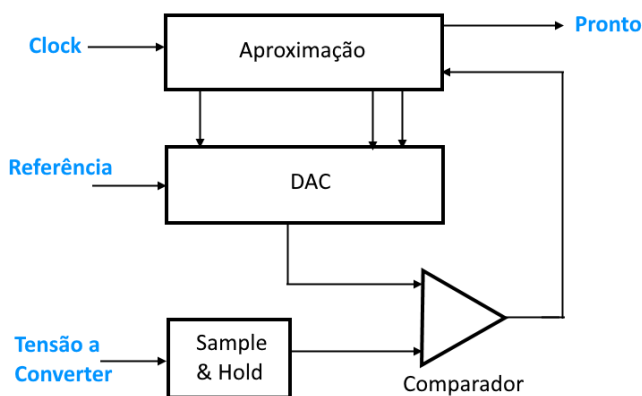


Figura 1 – Diagrama de blocos de um circuito conversor AD de aproximação sucessivas - <http://dqsoft.blogspot.com>

O circuito **Sample & Hold** amostra a tensão a ser convertida mantendo-a constante durante o tempo de conversão. Essa tensão é aplicada a uma das entradas de um comparador. A outra entrada do comparador recebe uma tensão de referência proveniente da saída de um conversor Digital Analógico – DAC. O circuito de



aproximação atua sobre o DAC mudando sucessivamente os valores digitais na entrada do DAC. Assim, por exemplo, quando o valor analógico de saída do DAC for maior que a tensão amostrada, a saída do comparador será ativada indicando que o valor digital presente no DAC atingiu o valor mais próximo equivalente ao valor de tensão amostrada na outra entrada do comparador. Portanto, para cada valor na entrada do DAC resultará em um bit na saída do comparador. Como o DAC irá variar sucessivamente os valores de entrada em busca do valor a ser convertido, a quantidade de bits do conversor impactará fortemente no tempo de conversão desse conversor AD.

O STM32F411 possui um ADC de 12 bits de aproximação sucessiva. Ele tem até 19 canais multiplexados, permitindo medir sinais de **16 fontes externas**, duas fontes internas e o canal VBAT. A conversão AD dos canais pode ser realizada em **modo único ou simples conversão, contínuo, varredura ou descontínuo**. O resultado do ADC é armazenado em um registro de dados de 16 bits alinhado à esquerda ou à direita.

No modo de **simples conversão** (single mode), o ADC é configurado para converter um único canal especificado pelo usuário. O ADC converte o sinal presente nesse canal e armazena o resultado no registro de dados. Após a conversão, o ADC para de funcionar até que uma nova conversão seja iniciada pelo usuário.

No modo de **conversão contínua**, o ADC começa a converter continuamente as entradas dos canais especificados, sem a necessidade de intervenção do usuário. A cada conversão, o resultado é armazenado no registro de dados do ADC.

Quando o ADC é configurado para o modo de **conversão contínua**, ele começa a converter continuamente as entradas desses canais. O resultado de cada conversão é armazenado no registro de dados de 16 bits alinhado à esquerda ou à direita, dependendo da configuração do usuário.

O modo de conversão contínua pode ser útil em aplicações onde os sinais que estão sendo medidos estão em constante mudança ou onde é necessário monitorar continuamente um grande número de canais. No entanto, é importante lembrar que a conversão contínua pode exigir mais tempo de processamento e energia do que outros modos de conversão.

No modo de conversão em varredura (scan mode), o ADC é configurado para converter um conjunto específico de canais em sequência, em vez de converter todos os canais continuamente. O usuário especifica a ordem em que os canais devem ser convertidos.

Ao configurar o ADC para o modo de varredura, o usuário especifica quais canais devem ser convertidos e em que ordem. Por exemplo, o usuário pode configurar o ADC para converter primeiro o canal 0, depois o canal 1 e assim por diante até o canal n.



Após a conversão de todos os canais na sequência especificada, o resultado é armazenado no registro de dados do ADC. O modo de **conversão em varredura** é útil em aplicações onde é necessário medir uma grande quantidade de sinais de forma organizada, e onde a ordem de conversão dos canais é importante.

É importante notar que a taxa de conversão do ADC pode ser afetada pelo modo de conversão em varredura, já que é necessário algum tempo para mudar de um canal para o próximo. Portanto, a taxa de conversão pode ser menor do que no modo de conversão contínua.

No modo de **conversão descontínua** (discontinuous mode), o ADC é configurado para converter apenas um canal de cada vez, e a conversão é realizada em um momento específico. Nesse modo, o usuário deve especificar qual canal deseja converter e quando a conversão deve ocorrer. Por exemplo, o usuário pode configurar o ADC para converter o canal 0 quando um sinal de disparo externo for recebido.

Quando ocorre a conversão, o resultado é armazenado no registro de dados do ADC. O modo de conversão descontínua é útil em aplicações onde é necessário converter canais específicos em momentos específicos, e onde a precisão é mais importante do que a velocidade de conversão.

É importante notar que, no modo de conversão descontínua, a taxa de conversão é geralmente menor do que no modo de conversão contínua ou no modo de varredura, pois o ADC converte apenas um canal de cada vez. No entanto, a precisão pode ser maior devido ao tempo extra disponível para cada conversão.

A forma de alinhamento dos dados do ADC pode ser feito de duas maneiras: a esquerda (left-aligned) e a direita (right-aligned). Por exemplo, com **alinhamento à direita**, se o ADC retorna uma conversão de **0x3F4**, o registro de dados ficará da seguinte maneira:

Bit: 15 ... 8 | 7 ... 0
Dados: 0000 0011 | 1111 0100
 03 **F4**

Se for **alinhamento à esquerda** o registro de dados ficará assim:

Bit: 15 ... 8 | 7 ... 0
Dados: 1111 0100 | 0000 0011
 F4 **03**

Observe que o alinhamento é realizado nos bytes do dado de 16 bits e não nos bits individualmente.

2. Conversor ADC na placa NÚCLEO

Nesse experimento usaremos a porta PA1 como entrada analógica para realizar simples aquisição e aquisições contínuas dos sinais analógicos para entrada do ADC. Esse pino é



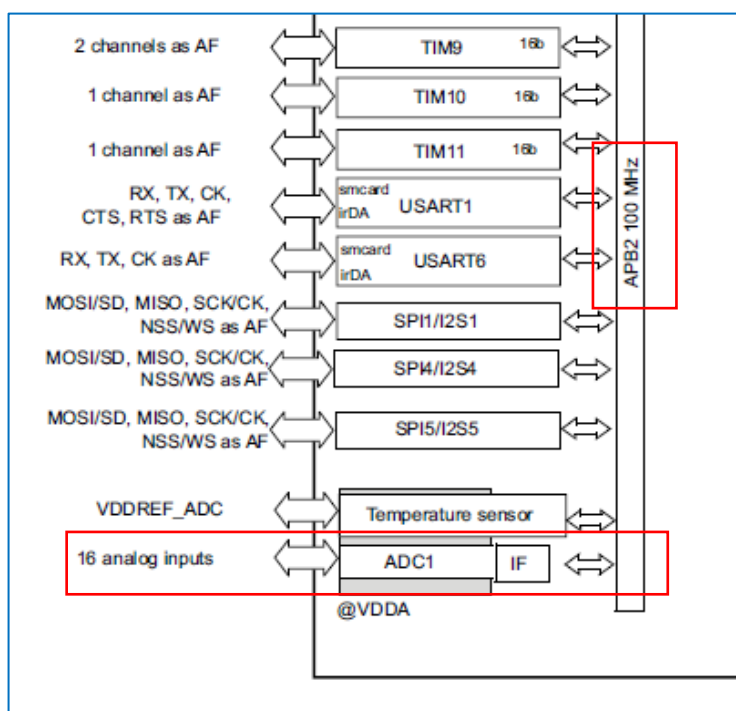
facilmente acessível na placa NÚCLEO através do conector MORPHO ou do conector ARDUINO®.

3. Implementação Prática

O ADC do STM32F411 possui 16 canais de entrada ADCx_IN[15:0]. O ADC é ligado setando o bit ADON no registro ADC_CR2. Quando o bit ADON é setado pela primeira vez, ele acorda o ADC do modo de economia de energia. A conversão começa quando o bit SWSTART ou o bit JSWSTART são setados. Você pode parar a conversão e colocar o ADC no modo de economia de energia limpando o bit ADON.

Os procedimentos para configuração do ADC e a leitura analógica da entrada PA1 no modo de simples conversão e conversão contínua são detalhados a seguir:

- ✓ Habilitar o clock para acesso ao GPIOA – RCC->[AHB1ENR](#) ;
- ✓ Configurar o modo de PA1 para entrada analógica - GPIOA->[MODER](#);
- ✓ Habilita o clock para acessar o ADC - RCC->[APB2ENR](#);



- ✓ Iniciar a sequência de conversão para o canal1 (PA1) - ADC1->[SQR3](#);

11.12.11 ADC regular sequence register 3 (ADC_SQR3)

Address offset: 0x34

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved				SQ6[4:0]				SQ5[4:0]				SQ4[4:1]			
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
SQ4_0		SQ3[4:0]				SQ2[4:0]				SQ1[4:0]					
r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w

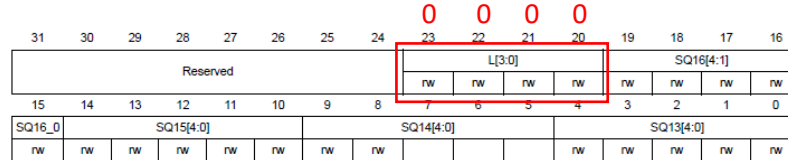


- ✓ Configura o tamanho (número de conversões) da sequência de Conversão - ADC1->**SQR1**;

11.12.9 ADC regular sequence register 1 (ADC_SQR1)

Address offset: 0x2C

Reset value: 0x0000 0000



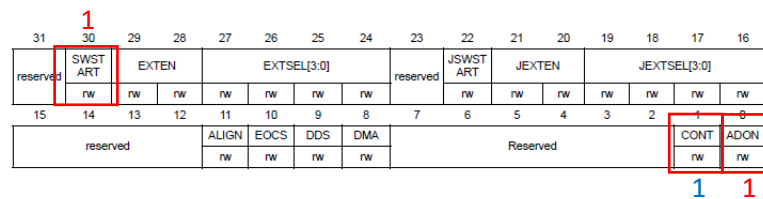
- ✓ Habilitar o módulo setando o bit ADON e Iniciar a conversão setando o bit SWSTART em ADC1->**CR2**;

Na **parte 2** do experimento o bit **CONT** deverá ser setado para o modo de aquisição contínua;

11.12.3 ADC control register 2 (ADC_CR2)

Address offset: 0x08

Reset value: 0x0000 0000

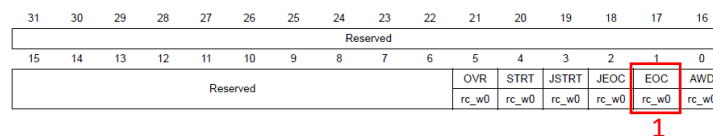


- ✓ Após o início da conversão, esperar o conversão terminar testando se o bit EOC em ADC1->**SR** está setado;

11.12.1 ADC status register (ADC_SR)

Address offset: 0x00

Reset value: 0x0000 0000



- ✓ Ler o resultado da conversão no registrador de dados DR - ADC1->**DR**;
- ✓ Para habilitação da interrupção do conversor quando finalizada a conversão é necessário setar o bit EOCIE no registrador **ADC_CR1**. Essa etapa será feita na PARTE3 da prática.

PARTE 1 – Conversão AD com simples aquisição:

- 1- Abra o STM32CUBEIDE. Faça uma cópia do 3_usart_serial. Renomeie o projeto para 4_AD_Converter;
- 2- Crie um novo arquivo fonte e edite o código salvando-o com o nome adc.c de acordo com o código a seguir:



adc.c

```
1 #include "stm32f4xx.h"
2 #include "adc.h"
3
4 #define GPIOAEN      (1U<<0)
5 #define ADC1EN        (1U<<8)
6 #define ADC_CH1        (1U<<0)
7 #define ADC_SEQ_LEN_1  0x00
8 #define CR2_AD0N       (1U<<0)
9 #define CR2_SWSTART    (1U<<30)
10 #define SR_EOC         (1U<<1)
11
12 void pa1_adc_init(void)
13 {
14     /**Configura o pino GPIO do ADC **/
15
16     /*Habilita o clock para acessar o GPIOA*/
17     RCC->AHB1ENR |= GPIOAEN;
18
19     /*Configura o modo de PA1 para analógico*/
20     GPIOA->MODER |= (1U<<2);
21     GPIOA->MODER |= (1U<<3);
22
23     /**Configura o modul ADC**/
24     /*Habilita o clock para acessar o ADC */
25     RCC->APB2ENR |= ADC1EN;
26
27     /*Inicia a sequência de conversão*/
28     ADC1->SQR3 = ADC_CH1;
29
30     /*Configura o tamanho da sequência de Conversão*/
31     ADC1->SQR1 = ADC_SEQ_LEN_1;
32
33     /*Habilita o módulo ADC*/
34     ADC1->CR2 |= CR2_AD0N;
35 }
```

```
36
37 void start_conversion(void)
38 {
39     /*Inicia a conversão AD*/
40     ADC1->CR2 |= CR2_SWSTART;
41 }
42
43 uint32_t adc_read(void)
44 {
45     /*Espera a conversão ter terminada*/
46     while(!(ADC1->SR & SR_EOC)){ }
47
48     /*Ler o resultado da conversão*/
49     return (ADC1->DR);
50 }
```

adc.c

- 3- Crie um novo arquivo cabeçalho (header) e edite o código salvando-o com o nome adc.h de acordo com o código a seguir:

```
1 #ifndef ADC_H_
2 #define ADC_H_
3
4 #include <stdint.h>
5
6 void pa1_adc_init(void);
7 uint32_t adc_read(void);
8 void start_conversion(void);
9
10 #endif
11
```

adc.h

4. Verifique se os códigos do módulo serial estão conforme descritos abaixo:



```
1 #include "uart.h"
2
3 #define GPIOAEN      (1U<<0)
4 #define UART2EN      (1U<<17)
5
6 #define CR1_TE        (1U<<3)
7 #define CR1_RE        (1U<<2)
8
9 #define CR1_UE        (1U<<13)
10 #define SR_TXE        (1U<<7)
11 #define SR_RXNE        (1U<<5)
12
13 #define SYS_FREQ      16000000
14 #define APB1_CLK      SYS_FREQ
15
16 #define UART_BAUDRATE 115200
17
18 static void uart_set_baudrate(USART_TypeDef *USARTx, uint32_t PeriphClk, uint32_t BaudRate);
19 static uint16_t compute_uart_bd(uint32_t PeriphClk, uint32_t BaudRate);
20
21 void uart2_write(int ch);
22
23 int __io_putchar(int ch)
24 {
25     uart2_write(ch);
26     return ch;
27 }
28
29 void uart2_rtxx_init(void)
30 {
31     /* Configura o pino gpio da uart */
32     /* Habilita o clock para acessar gpioa */
33     RCC->AHB1ENR |= GPIOAEN;
34
35     /* Configura PA2 para modo alternate function */
36     GPIOA->MODER &= ~(1U<<4);
37     GPIOA->MODER |= (1U<<5);
38
39     /* Configura PA2 para tipo alternate function */
40     GPIOA->AFR[0] |= (1U<<8);
41     GPIOA->AFR[0] |= (1U<<9);
42     GPIOA->AFR[0] |= (1U<<10);
43     GPIOA->AFR[0] &= ~(1U<<11);
44
45     /* Configura PA3 para modo alternate function */
46     GPIOA->MODER &= ~(1U<<6);
47     GPIOA->MODER |= (1U<<7);
48
49     /* Configura PA3 para tipo alternate function */
50     GPIOA->AFR[0] |= (1U<<12);
51     GPIOA->AFR[0] |= (1U<<13);
52     GPIOA->AFR[0] |= (1U<<14);
53     GPIOA->AFR[0] &= ~(1U<<15);
54
55     /* Configura o módulo uart */
56     /* Habilita o clock para acessar a uart2 */
57     RCC->APB1ENR |= UART2EN;
58
59     /* Configura o baudrate */
60     uart_set_baudrate(USART2, APB1_CLK, UART_BAUDRATE);
61
62     /* Habilita a transmissão e recepção serial */
63     USART2->CR1 = (CR1_TE | CR1_RE);
64
65     /* Habilita o módulo uart */
66     USART2->CR1 |= CR1_UE;
67 }
68
69 /* Função que leu um byte pela serial */
70 char uart2_read(void)
71 {
72     /* Testa se o registrador de recepção de dados não está vazio */
73     while(!(USART2->SR & SR_RXNE)){}
74
75     /* Ler o dado recebido pela serial */
76     return USART2->DR;
77 }
78
79 /* Função que transmite um byte pela serial */
80 void uart2_write(int ch)
81 {
82     /* Testa se o registrador de transmissão de dados está vazio */
83     while(!(USART2->SR & SR_TXE)){}
84
85     /* Escreve no registrador de dados */
86     USART2->DR = (ch & 0xFF);
87 }
88
89 static void uart_set_baudrate(USART_TypeDef *USARTx, uint32_t PeriphClk, uint32_t BaudRate)
90 {
91     USARTx->BRR = compute_uart_bd(PeriphClk, BaudRate);
92 }
93
94 static uint16_t compute_uart_bd(uint32_t PeriphClk, uint32_t BaudRate)
95 {
96     return ((PeriphClk + (BaudRate/2U))/BaudRate);
97 }
98
99 }
```

```
1 #ifndef UART_H_
2 #define UART_H_
3 #include <stdint.h>
4
5 #include "stm32f4xx.h"
6
7 char uart2_read(void);
8 void uart2_rtxx_init(void);
9
10 #endif /* UART_H_ */
11
```

uart.h



5. Edite o arquivo main.c conforme descrito abaixo:

```
1 #include <stdio.h>
2 #include <stdint.h>
3 #include "stm32f4xx.h"
4 #include "uart.h"
5 #include "adc.h"
6
7 int sensor_value;
8
9 int main(void)
10 {
11     uart2_rtx_init();
12     pa1_adc_init();
13
14     while(1)
15     {
16         start_conversion();
17         sensor_value = adc_read();
18         printf("valor digital lido = %d \n", sensor_value);
19         for(int i=0; i<100000; i++){ }
20     }
21     return 0;
22 }
23
```

main.c

6. Compile e carregue o código na placa Núcleo;
7. Faça o DEBUG do código e adicione a variável **sensor_value** à lista de variáveis que podem ser observadas em tempo de execução (Live Expressions).

(x)= Variables Breakpoints Expressions Registers Live Expressions SFRs		
Expression	Type	Value
(x)= sensor_value	int	521
+ Add new expression		

8. Observe o resultado no terminal serial HERCULES;
9. Adicione um potenciômetro de forma a variar a tensão na entrada PA1 e observe o resultado no valor da variável no Live Expressions e os dados recebidos no terminal HERCULES.

CONTINUA ...



PARTE 2 – Conversão AD em modo contínuo:

1. Modifique o arquivo `adc.c` adicionando as instruções conforme discriminado abaixo:

```
9  #define CR2_SWSTART    (1U<<30)
10 #define SR_EOC         (1U<<1)
11 #define CR2_CONT       (1U<<1)

38 void start_conversion(void)
39 {
40     /* Habilita a conversão contínua */
41     ADC1->CR2 |= CR2_CONT;
42     /* Inicia a conversão AD */
43     ADC1->CR2 |= CR2_SWSTART;
44 }
45
```

adc.c

2. Faça o DEBUG do código e adicione a variável `sensor_value` à lista de variáveis que podem ser observadas em tempo de execução (Live Expressions);
3. Observe o resultado no terminal serial HERCULES;
4. Varie a tensão na entrada PA1 e observe o resultado no valor da variável no Live Expressions e os dados recebidos no terminal HERCULES.

CONTINUA ...



PARTE 3 – Conversão AD com interrupção:

1. Modifique o arquivo adc.c e main.c adicionando as instruções conforme discriminado abaixo:

```
adc.c
1 #include "stm32f4xx.h"
2 #include "adc.h"
3
4 #define GPIOAEN (1U<<0)
5 #define ADC1EN (1U<<8)
6 #define ADC_CH1 (1U<<0)
7 #define ADC_SEQ_LEN_1 0x00
8 #define CR2_ADON (1U<<0)
9 #define CR2_SWSTART (1U<<30)
10 #define CR2_CONT (1U<<1)
11 #define CR1_EOCIE (1U<<5)
12
13 void pa1_adc_interrupt_init(void)
14 {
15     /*Configura o pino GPIO do ADC */
16
17     /* Habilita o clock para acessar o GPIOA */
18     RCC->AHB1ENR |= GPIOAEN;
19
20     /* Configura o modo de PA1 para analógico */
21     GPIOA->MODER |= (1U<<2);
22     GPIOA->MODER |= (1U<<3);
23
24     /*Configura o módulo ADC*/
25     /*Habilita o clock para acessar o ADC */
26     RCC->APB2ENR |= ADC1EN;
27
28     /* Habilita o fim da conversão por interrupção */
29     ADC1->CR1 |= CR1_EOCIE;
30     /* Habilita a interrupção do ADC no NVIC */
31     NVIC_EnableIRQ(ADC_IRQn);
32
33     /*Inicia a sequência de conversão */
34     ADC1->SQR3 = ADC_CH1;
35
36     /*Configura o tamanho da sequência de Conversão*/
37     ADC1->SQR1 = ADC_SEQ_LEN_1;
38
39     /*Habilita o módulo ADC*/
40     ADC1->CR2 |= CR2_ADON;
41 }
42
main.c
1 #include <stdio.h>
2 #include <stdint.h>
3 #include "stm32f4xx.h"
4 #include "uart.h"
5 #include "adc.h"
6
7 static void adc_callback(void);
8
9 uint32_t sensor_value;
10
11 int main(void)
12 {
13     uart2_tx_init();
14     pa1_adc_interrupt_init();
15     start_conversion();
16     while(1)
17     {
18         // nada
19     }
20 }
21
22 static void adc_callback(void)
23 {
24     sensor_value = ADC1->DR;
25     printf("Sensor value : %d \n\r", (int)sensor_value);
26     for(int i =0; i<100000; i++){
27     }
28 }
29 void ADC_IRQHandler(void)
30 {
31     /* Verifica o bit EOC - Fim da conversão */
32     if((ADC1->SR & SR_EOC) !=0)
33     {
34         /* Limpa o EOC*/
35         ADC1->SR &=~SR_EOC;
36         adc_callback();
37     }
38 }
```

2. Faça o DEBUG do código e adicione a variável **sensor_value** à lista de variáveis que podem ser observadas em tempo de execução (Live Expressions);
3. Observe o resultado no terminal serial HERCULES;
4. Varie a tensão na entrada PA1 e observe o resultado no valor da variável no Live Expressions e os dados recebidos no terminal HERCULES.

CONTINUA ...



TAREFA1 [5.0]:

Modifique o experimento para que seja enviada pela serial uma simples conversão do ADC ao pressionar o push button da placa NÚCLEO. A detecção do final de conversão deve ser feita por interrupção.

TAREFA2 [5.0]:

Modifique o experimento anterior de forma que o valor digital lido seja convertido em **VOLTS** e enviado pelo terminal serial.

OBS: O valor lido, variável **sensor_value**, agora deve ser um **float**. Nesse caso, é necessário habilitar os flags para uso do **printf** com **float**, conforme mostrado abaixo:

